P. R. POTE PATIL

College of Engineering & Management, Amravati.

(An Autonomous Institute)

Department of Artificial Intelligence & Data Science

Year: 2025-26 **Semester: IV**

Course: Node.JS Lab (AI409VSE04)

Institute Vision Mission

Vision

To flourish as a center of excellence for producing the skilled technocrats and committed human beings.

Mission

- To create conducive environment for teaching &learning.
- To impart quality education through demanding academic programs.
- To enhance career opportunities by exposure to Industries & recent technologies.
- To develop professionals with strong ethics and human values for the betterment of society.

Department of Artificial Intelligence & Data Science

Vision

To flourish as a center of excellence for producing the skilled of Artificial Intelligence and committed human beings.

Mission

- To impart quality Artificial Intelligence engineering education through demanding academic and skilled based programs
- To enhance career opportunities in Artificial Intelligence by exposure to Industries & recent technologies
- To develop professionals with strong ethics and human values for the betterment of Society.

Program Outcomes

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8)

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7)

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the Engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i)independent and life-long learning ii) adaptability to new and emerging technologies and iii)critical thinking in the broadest context of technological change. (WK8)

Program Education Objectives

To prepare the Engineering graduates to,

PEO1: Technical Proficiency: Graduates will demonstrate proficiency in core areas of computer science, including algorithms, data structures, software engineering, and database systems, with a specialized focus on artificial intelligence and machine learning techniques.

PEO2: Leadership and Innovation: Graduates will exhibit leadership qualities and innovative thinking, contributing to the development and implementation of AI-driven solutions that address societal, industrial, and environmental challenges.

PEO3: Ethical and Professional Responsibility: Graduates will adhere to ethical and professional standards in their engineering practice, demonstrating integrity, accountability, and a commitment to societal well-being in the design and deployment of AI and DS systems.

PEO4: Lifelong Learning: Graduates will adopt a culture of lifelong learning and professional development, empowering individuals to adapt to emerging technologies, navigate complex socio-technical landscapes, and contribute meaningfully to the advancement of AI&DS throughout their careers.

Program Specific Outcomes

To prepare the Engineering graduates to,

PSO1: Intelligent Systems Development: Develop intelligent systems and applications integrating AI and DS technologies to enhance functionality and performance.

PSO2: Problem-Solving Skills: Possess strong analytical and problem-solving skills, enabling them to identify, formulate, and solve complex engineering problems using AI and DS methodologies in diverse application domains.

GUIDELINES FOR TEACHERS

Teachers shall discuss the following points with students before start of practical of the subject.

1. Learning Overview: To develop better understanding of importance of the subject. To know related skills to be developed such as intellectual and motor skills.
2. Know your Laboratory Work: To understand the layout of laboratory, specifications of equipment / instruments /materials, procedure, working in groups, planning time etc. also to know total amount of work to be done in the laboratory.
3. Teacher shall ensure that required equipment is in working condition before start of each experiment, also keep operating instruction manual available.
4. Explain prior concepts to the students before starting of each experiment.
5. Evolve student's activity at the time of conduct of each experiment.
6. While taking reading / observation each student (from batch of 20 students) shall be given a chance to perform / observe the experiment.
7. Teacher shall assess the performance of students continuously.
8. Teacher is expected to share the skills to be developed in the students.
9. Teacher should ensure that the respective skills are developed in the students after the completion of the practical exercise.
10. Teacher may provide additional knowledge and skills to the students even though not covered in the manual but are expected from students by the industries.
11. Teacher may suggest the students to refer additional related literature of the technical papers / reference books / Seminar Proceedings, etc.
12. Focus should be given on development of enlisted skills rather than theoretical / codified knowledge.
13. During assessment teacher is expected to ask questions to the students to tap their achievements regarding related knowledge and skills.
14. Teacher should give more focus on hands on skills.

INSTRUCTIONS FOR STUDENTS

1. Students shall read the points given below for understanding the theoretical concepts and practical applications.
2. Listen carefully to the lecture given by teacher about importance of subject, curriculum philosophy, learning structure, skills to be developed, information about equipment, instruments, procedure, method of continuous assessment, tentative plan of work in laboratory and total amount of works to be done in a semester.
3. Student shall undergo study visit of the laboratory for types of equipment, and material to be used, before performing experiments.
4. Read the write up of each experiment to be performed, a day in advance.
5. Organize the work in the group and make a record of all observations.
6. Understand the purpose of experiment and its practical implications.
7. Student should not hesitate to ask any difficulty faced during conduct of practical /exercise.
8. Write the answers of the questions allotted by the teacher during practical hours if possible or afterwards, but immediately.
9. Student should develop the habit of pear discussion / group discussion related to experiments / exercise so that exchanges of knowledge / skills could take place.
10. Students shall attempt to develop related hands-on-skills and gain confidence.
11. Student shall focus on development of skills rather than theoretical or codified knowledge.
12. Student shall insist for the completions of recommended Laboratory Work, answers to the given question etc.
13. Student shall develop the habit of evolving more ideas, innovations, skills etc. that included in the scope of the manual.
14. Student shall refer technical magazines, proceedings of the Seminars, refer website related to the scope of the subjects and update their knowledge and skills.
15. Student should develop the habit of not depend totally on teachers but to develop self-learning techniques.
16. Student should develop the habit to interact with the teacher without hesitation with respect to academic involved.
17. Student should develop habit to submit the practical's exercise continuously and progressively on the schedule dates and should get the assessment done.
18. Student should be well prepared while submitting the write up of the exercise. This will develop the continuity of the studies and he will not be overloaded at the end.

P. R. POTE PATIL

COLLEGE OF ENGINEERING & MANAGEMENT, AMRAVATI.

Department of Artificial Intelligence & Data Science

Certificate

This is to certify that Mr./ Ms..... of

…….. Semester of Bachelor of Technology in Artificial Intelligence &

Data Science of P. R. Pote Patil College of Engineering &

Management, Amravati, has completed the term work satisfactory of

course …….. for the academic year 20 - 20 as

prescribed in the curriculum.

Date…… **Roll No……..**

Subject Teacher **Head of the Department**

LIST OF (PRACTICALS / EXPERIMENTS) & PROGRESSIVE ASSESSMENT FOR TERM WORK

Academic Year :2025-26 **Course** ……

Course Code: (AI409VSE04) **Semester: IV**

Name of Faculty: ……

Name of Student: ……

Roll No ……

SN.	Title of the Practical / Experiment	Date of Performance	Date of Submission	Sign of Teacher
1	Introduction to Node.js and its Installation			
2	Demonstrate the Built-in Module in Node.js			
3	Create the HTTP module and how Node.js act as a server			
4	Implement Established Node.js file system module to read, write, create, update, delete and rename the file.			
5	Write a program that opens the requested file and returns the content of the file to the client. If anything goes wrong throw a 404 error.			
6	Explore the npm package axios to Fetch Data from an API			
7	Demonstrate the Event handling in Node.js and demonstrate it using event module.			
8	Explore the Concept of Formidable module in Node.js to upload the file on the server			
9	Implement Event Emitter Object in Node.js			
10	Create Node mailer module and ready to send mail on the server.			
11	Implement the database (MongoDB/MySQL) connection with Node.js			
12	Write a Node.js program to read, parse, modify, and write data in a JSON file using the File System (fs) module			

Signature of Faculty

Course Outcomes

After successful completion of laboratory course, the students will able to

SN	Outcomes	BT Level	PO
1	Study of Node.js as back end environment	1	1,2,5
2	Implement built-in modules of Node.js as server	2	2,5,10,11
3	ToapplyeventhandlingmoduleinNode.js	1,2	5,10,11

I. Rubrics used for continuous Assessment in every lab session

Skills	Allocated Marks	High	Medium	Low
		10-08	07-04	03-00
Process Related Skills	**10**	Student performed or executed experiments, obtained results, and drew conclusions fully as per expectation within a due date.	Student performed or executed experiments, obtained results, and drew conclusions partially as per expectation after a due date.	Student performed or executed experiments, obtained results, and drew conclusions below the expectation after two weeks of a due date.
Product Related Skills	**10**	**10-08**	**07-04**	**03-01**
		Records submitted by the student on due date and found highly satisfactory after evaluation.	Records submitted by the student after one week of due date and found moderately satisfactory after evaluation.	Records submitted by the student after two week of due date and found highly dissatisfactory after evaluation.
Viva Voce	**05**	**05-04**	**03-02**	**01**
		Student answered the Questions Extraordinarily.	Student answered the Qaestions satisfactorily.	Student answrer the Questions moderately satisfaetorily.
Total Marks for Lab Performance 25 Marks				

II. Rubrics used for External Assessment in Lab Course

Skills	Allocated Marks	Parameters	High	Medium	Low
Process Related Skills	**15**	Handle equipment/ tools/ commands correctly or logic formation and Work cohesively in team **(7)**	Most satisfactory **(07-06)**	Partially successful **(05-04)**	Below expectation **(03-01)**
		Integrate system correctly & measure parameters correctly or Debugging Ability **(8)**	Highly satisfactory **(08-07)**	Partially correct **(05-06)**	Incorrect or unsatisfactory **(04-01)**
Product Related Skills	**10**	Obtain correct results, Answer practical related questions **(5)**	Exceptional **(05-04)**	Satisfactory **(03)**	Unsatisfactory **(02-01)**
		Interpret results, draw conclusion **(5)**	Exceptional **(05-04)**	Satisfactory **(03)**	Unsatisfactory **(02-01)**
Total Marks		**25 Marks**			

Practical No.1

Aim: Exploration of the node.js and its installation to print hello world.

Objectives: 1. To introduce Node JS

2.To install Node JS

Theory: What is Node.js?

Node.js is a free, open-source JavaScript runtime that runs on Windows, Mac, Linux, and more.

It lets you execute JavaScript code outside of a web browser, enabling server-side development with JavaScript. Built on Chrome's V8 JavaScript engine, Node.js is designed for building scalable network applications efficiently.

- Node.js can generate dynamic page content
- Node.js can create, open, read, write, delete, and close files on the server
- Node.js can collect form data
- Node.js can add, delete, modify data in your database

- What is a Node.js File?

  - Node.js files contain tasks that will be executed on certain events.
  - A typical event is someone trying to access a port on the server.
  - Node.js files must be initiated on the server before having any effect.
  - Node.js files have extension ".js"

Installation:

The official Node.js website has installation instructions for Node.js https://nodejs.org

Node.js files must be initiated in the "Command Line Interface" program of your computer.

How to open the command line interface on your computer depends on the operating system. For Windows users, press the start button and look for "Command Prompt", or simply write "cmd" in the search field.

Navigate to the folder that contains the file "myfirst.js", the command line interface window should look something like this:

C:\Users*Your Name*>_

If anyone tries to access your computer on port 8080, they will get a "Hello World!" message in return!

Program

Output:

Conclusion:

Signature of Facult

Practical No.2

Aim: To implement built-in module in Node.js

Objective:

1. To understand built-in and user defined modules
2. To write server application which uses these modules

What is a Module in Node.js?

Consider modules to be the same as JavaScript libraries. A

set of functions you want to include in your application.

Built-in Modules

Node.js has a set of built-in modules which you can use without any further installation.

Module **Description**

Module	Description
assert	Provides a set of assertion tests. book.vips.systems
buffer	To handle binary data. book.vips.systems
child_process	To run a child process. book.vips.systems
cluster	To split a single Node process across CPUs. book.vips.systems
crypto	Cryptographic functions (OpenSSL). book.vips.systems
dgram	UDP datagram sockets. book.vips.systems
dns	DNS lookups and resolution. book.vips.systems
domain	Deprecated (was for handling unhandled errors). book.vips.systems
events	Event-handling (EventEmitter). book.vips.systems

fs	File system operations. book.vips.systems
http	HTTP server/client support. book.vips.systems
https	HTTPS server/client support. book.vips.systems
net	Networking (TCP server/client). book.vips.systems
os	OS information. book.vips.systems
path	File path utilities. book.vips.systems
punycode	Deprecated encoding scheme. book.vips.systems
querystring	URL query-string utilities. book.vips.systems
readline	Readable stream line-by-line handling. book.vips.systems
stream	Streaming data interfaces. book.vips.systems
string_decoder	Decode Buffer into strings. book.vips.systems
timers	Scheduling functions (setTimeout, etc). book.vips.systems
tls	TLS/SSL support. book.vips.systems
tty	Terminal interface. ROLANDO - MY SHARING BLOG
url	URL string parsing. ROLANDO - MY SHARING BLOG
util	Utility functions. ROLANDO - MY SHARING BLOG
v8	Access V8 engine info. ROLANDO - MY SHARING BLOG
vm	Virtual machine to compile/execute code. ROLANDO - MY SHARING BLOG
zlib	Compression/decompression. ROLANDO - MY SHARING BLOG

Program:

Output:

Conclusion:

Signature of Faculty

Practical no.3

Aim: Implement HTTP module and implement Node.js as a web server.

Objective:1) To understand the http module

2) To implement http module

Software used: Node.js

Theory:

The node.js framework is mostly used to create server based application.The framework can easily be used to create web server which can serve content to users. There are a varity of module such as the 'http' and 'request' module which helps in processing server related request in the web server space.we will have a look at how we can create a basic web server application using Node.js.

To access web pages of any web applications you need a web server.The web server wll handle all the http requests for the web application ex.IIS is a web server for ASP.NET web application and apache is a web server for PHP or Java web applications.Node.js web provides capabilities to create your own web server which will handle HTTP request Asynchronously. You can use IIS or Apache to run node.js web applications but it is recommended to use Node.js web server.

Program:

Output:

Conclusion:

Signature of Faculty

Practical No .4

Aim: Implement Node.js file system module to read, write, create, update, delete and rename the file.

Objectives:1)To understand file system module

2) To implement file system module

Software used: Node.js

Theory: The Node.js file system module allows you to work with the file system on your computer.

To include the File System module, use the require() method: var
fs = require('fs');

Common use for the File System module:

- Read files
- Create files
- Update files
- Delete files
- Rename files

Read Files

The fs.readFile()method is used to read files on your computer.

Assume we have the following HTML file (located in the same folder as Node.js):

demofile1.html

```
<html>
<body>
<h1>My Header</h1>
<p>My paragraph.</p>
</body>
</html>
```

Create a Node.js file that reads the HTML file, and return the content:

```
var http = require('http');
var fs = require('fs');
http.createServer(function (req, res) {
  fs.readFile('demofile1.html', function(err, data) {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.write(data);
    return res.end();
  });
}).listen(8080);
```

Save the code above in a file called "demo_readfile.js", and initiate the file:

Initiate demo_readfile.js:

C:\Users*Your Name*>node demo_readfile.js

Create Files

The File System module has methods for creating new files:

- fs.appendFile()
- fs.open()
- fs.writeFile()

The fs.appendFile()method appends specified content to a file. If the file does not exist, the file will be created:

Example

Create a new file using the appendFile() method:

```
var fs = require('fs');

fs.appendFile('mynewfile1.txt', 'Hello content!', function (err)
  { if (err) throw err;
  console.log('Saved!');
});
```

```
var fs = require('fs');

fs.appendFile('mynewfile1.txt', ' This is my text.', function (err)
  { if (err) throw err;
  console.log('Updated!');
});
»
```

Program:

Output:

Conclusion:

Signature of Faculty

Practical No.5

Aim: To study the URL module of the node.js and write a program that opens the requested file and returns the content of the file to the client.If anything goes wrong throw a 404 error.

Objectives: 1)To understand URL module of the node.js

2)To write a program that opens the requested file and returns the content of the file to client.

Software required: Node.js

The Built-in URL Module

The URL module splits up a web address into readable parts. To include the

URL module, use the require() method:

```
var url = require('url');
```

Parse an address with the url.parse() method, and it will return a URL object with each part of the address as properties:

Split a web address into readable parts:

```
var url = require('url');
var adr = 'http://localhost:8080/default.htm?year=2017&month=february'; var q = url.parse(adr,
true);

console.log(q.host); //returns 'localhost:8080' console.log(q.pathname); //returns
'/default.htm' console.log(q.search); //returns '?year=2017&month=february'

var qdata = q.query; //returns an object: { year: 2017, month: 'february' } console.log(qdata.month); //returns
'february'
```

Node.js File Server

Now we know how to parse the query string, and in the previous chapter we learned how to make Node.js behave as a file server. Let us combine the two, and serve the file requested by the client.

Create two html files and save them in the same folder as your node.js files.

summer.html

```
<!DOCTYPE html>
<html>
<body>
<h1>Summer</h1>
<p>I love the sun!</p>
</body>
</html>
```

winter.html

```
<!DOCTYPE html>
<html>
<body>
<h1>Winter</h1>
<p>I love the snow!</p>
</body>
</html>
```

Create a Node.js file that opens the requested file and returns the content to the client. If anything goes wrong, throw a 404 error:

demo_fileserver.js:

```
var http = require('http'); var url =
require('url'); var fs = require('fs');

http.createServer(function (req, res) { var  q  =
   url.parse(req.url,  true); var filename = "." +
   q.pathname;
   fs.readFile(filename, function(err, data) { if (err) {
         res.writeHead(404, {'Content-Type': 'text/html'}); return res.end("404
         Not Found");
      }
      res.writeHead(200, {'Content-Type': 'text/html'}); res.write(data);
```

```
        return res.end();
    });
}).listen(8080);
```

Program:

Output:

Conclusion:

Signature of Faculty

Practical No.6

Aim: Explore the npm package axios to Fetch Data from an API.

Theory: What is axios?

- **axios** is a popular Node.js package used to make **HTTP requests**.
- It works with **Promises**, which makes asynchronous code easier to manage.
- It supports:
  - GET, POST, PUT, DELETE requests
  - Handling JSON data
  - Error handling
  - Making API calls easily

Why use axios?

- Simple syntax (easier than Node's built-in http module)
- Automatically parses JSON responses
- Better error messages
- Works in both **Node.js and browsers**

Installing axios

Before writing code, install axios in your project:

npm install axios

This command downloads axios into your project's node_modules folder.

Below is a working code example to fetch data from a public API using axios:

```
const axios = require('axios');

// Example API URL
const apiURL = 'https://jsonplaceholder.typicode.com/posts/1';

// Using axios to fetch data
axios.get(apiURL)
  .then(response => {
```

```
console.log("Data received from API:");
    console.log(response.data);  // Prints JSON data
  })
  .catch(error => {
    console.log("Error occurred while fetching data:");
    console.log(error.message);
  });
```

Program:

Output:

Conclusion:

Signature of Faculty

Practical No.7

Aim: To study event handling in Node.js and demonstrate it using event module.

Software required: Node.js

Events in Node.js

Every action on a computer is an event. Like when a connection is made or a file is opened.

Objects in Node.js can fire events, like the readStream object fires events when opening and closing a file:

Example Get your own Node.js Server

```
var fs = require('fs');
var rs = fs.createReadStream('./demofile.txt');
rs.on('open', function () {
  console.log('The file is open');
});
```

Events Module

Node.js has a built-in module, called "Events", where you can create-, fire-, and listen for- your own events.

Program:

Output:

Conclusion:

Signature of Faculty

Practical No.8

Aim: To Study & Implement the Formidable module of Node.js to upload the file on the server.

The Formidable Module

There is a very good module for working with file uploads, called "Formidable".

The Formidable module can be downloaded and installed using NPM:

```
C:\Users\Your Name>npm install formidable
```

After you have downloaded the Formidable module, you can include the module in any application:

```
var formidable = require('formidable');
```

Upload Files

Now you are ready to make a web page in Node.js that lets the user upload files to your computer:

Step 1: Create an Upload Form

Create a Node.js file that writes an HTML form, with an upload field:

ExampleGet your own Node.js Server

This code will produce an HTML form:

```
var http = require('http');

http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.write('<form action="fileupload" method="post" enctype="multipart/form-data">');
  res.write('<input type="file" name="filetoupload"><br>');
  res.write('<input type="submit">');
  res.write('</form>');
  return res.end();
}).listen(8080);
```

Step 2: Parse the Uploaded File

Include the Formidable module to be able to parse the uploaded file once it reaches the server.

When the file is uploaded and parsed, it gets placed on a temporary folder on your computer.

Example

The file will be uploaded, and placed on a temporary older:

```
var http = require('http');
var formidable = require('formidable');

http.createServer(function (req, res) {
  if (req.url == '/fileupload') {
    var form = new formidable.IncomingForm();
    form.parse(req, function (err, fields, files) {
    res.write('File uploaded');
      res.end();
    });
  } else {
    res.writeHead(200, {'Content-Type': 'text/html'});
    res.write('<form action="fileupload" method="post" enctype="multipart/form-data">');
    res.write('<input type="file" name="filetoupload"><br>');
    res.write('<input type="submit">');
    res.write('</form>');
    return res.end();
  }
}).listen(8080);
```

Program:

Output:

Conclusion:

Signature of Faculty

Practical No.9

Aim: Implement Event Emitter Object in Node.js

The Event-Emitter Object

You can assign event handlers to your own events with the Event-Emitter object.

In the example below we have created a function that will be executed when a "scream" event is fired.

To fire an event, use the `emit()` method.

Program:

Output:

Conclusion:

Signature of Faculty

Practical No.10

Aim: Create Node mailer module and ready to send mail on the server.

The Nodemailer Module

The Nodemailer module makes it easy to send emails from your computer.

The Nodemailer module can be downloaded and installed using npm:

```
C:\Users\Your Name>npm install nodemailer
```

After you have downloaded the Nodemailer module, you can include the module in any application:

```
var nodemailer = require('nodemailer');
```

Send an Email

Now you are ready to send emails from your server.

Use the username and password from your selected email provider to send an email. This tutorial will show you how to use your Gmail account to send an email:

Example[Get your own Node.js Server]

```
var nodemailer = require('nodemailer');

var transporter = nodemailer.createTransport({
  service: 'gmail',
  auth: {
    user: 'youremail@gmail.com',
    pass: 'yourpassword'
  }
});

var mailOptions = {
  from: 'youremail@gmail.com',
  to: 'myfriend@yahoo.com',
```

```
  subject: 'Sending
  Email using Node.js',
  text: 'That was easy!'
};

transporter.sendMail(mailOptions,
  function(error, info){ if (error) {
  console.log(error);
  } else {
    console.log('Email sent: ' + info.response);
  }
});
```

Program:

Output:

Conclusion:

Signature of Faculty

Practical No.11

Aim: Implement the database (MongoDB/MySQL) connection with Node.js

Theory: What is Database Connectivity in Node.js?

Database connectivity means connecting a Node.js application to a database server (MySQL or MongoDB) so that the application can:

- Store data
- Retrieve data
- Update data
- Delete data

Node.js uses external **npm packages** to communicate with databases.

2 MySQL Database Connection with Node.js
Required Package
npm install mysql
Code: Connect Node.js to MySQL

```
const mysql = require('mysql');

// Create connection
const connection = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: '',
  database: 'studentdb'
});

// Connect to MySQL
connection.connect(function (err) {
if (err) {
    console.log("Connection Failed!");
    throw err;
  }
  console.log("Connected to MySQL Database!");
});
```

MongoDB Database Connection with Node.js
Required Package
npm install mongoose

What is mongoose?

- A MongoDB ODM (Object Data Modeling)
- Makes working with MongoDB easier
- Helps define schemas and models

Code: Connect Node.js to MongoDB

```
const mongoose = require('mongoose');

// MongoDB connection URL
const url = "mongodb://127.0.0.1:27017/studentdb";

// Connect to MongoDB
mongoose.connect(url)
  .then(() => {
    console.log("Connected to MongoDB Database!");
  })
  .catch((err) => {
    console.log("Connection Error:", err);
  });
```

Program:

Output:

Conclusion:

Signature of Faculty

Practical No.12

Aim: Write a Node.js program to read, parse, modify, and write data in a JSON file using the File System(fs)module.

Theory: What is the FS (File System) Module?

- The fs module in Node.js allows you to interact with the file system.
- You can read, write, update, delete, and manage files.

What is JSON?

- JSON (JavaScript Object Notation) is a lightweight data format used to store structured data.
- Node.js can easily handle JSON since it is JavaScript-based.

Example JSON File (data.json)

Create a file named data.json:

```
{
  "name": "John",
  "age": 25,
  "city": "Delhi"
}
```

Node.js Program (Using fs Module)

```
const fs = require('fs');

// Step 1: Read JSON file

fs.readFile('data.json', 'utf8', (err, data) => {
  if (err) {
    console.log("Error reading file:", err);
    return;
  }

  // Step 2: Parse JSON data

  let jsonData = JSON.parse(data);
  console.log("Original Data:", jsonData);

  // Step 3: Modify JSON data

  jsonData.age = 30;                    // update value
  jsonData.country = "India";               // add new key-value pair

  // Step 4: Convert back to JSON string

  const newData = JSON.stringify(jsonData, null, 2);

  // Step 5: Write updated data to file

  fs.writeFile('data.json', newData, (err) => {
    if (err) {
      console.log("Error writing file:", err);
        return;
      }
      console.log("File updated successfully!");
  });
});
```

Program:

Output:

Conclusion:

Signature of Faculty